STUDENT TEACHER ACADEMIC PORTAL

Submitted by

UDAYAKUMAR D

1P24MC056

In partial fulfillment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

From Bharathiar University, Coimbatore.

Under the internal supervision of

J.GNANA MANO SHEEBA, M.C.A., M. Phil., B.Ed.,

Assistant Professor

SCHOOL OF COMPUTER STUDIES

MASTER OF COMPUTER APPLICATIONS

RVS COLLEGE OF ARTS AND SCIENCE (AUTONOMOUS)

Sulur, Coimbatore – 641402.

March - 2026

RVS COLLEGE OF ARTS AND SCIENCE (AUTONOMOUS)

SCHOOL OF COMPUTER STUDIES -MCA

Master Of Computer Applications

March- 2026

Register Number: 1P24MC056

Certified bona fide Project Work done by UDAYAKUMAR D

Guide **Director**

Submitted for the Project Evaluation and *Viva voce* held on _____

Internal Examiner External Examiner

Certificate

Date:

TO WHOM IT MAY CONCERN

This is to certify that the project work titled "**STUDENT TEACHER ACADEMIC PORTAL"** using MERN Stack is a Bonafide work of **UDAYAKUMAR D (1P24MC056).** He has completed the In-house project work at Department of MCA, RVS College of Arts and Science, Coimbatore during the period from Nov 2025 to Mar 2026 towards the partial fulfilment of the requirement for the award of MCA degree prescribed by Bharathiar University, Coimbatore-46.

Director

CERTIFICATE

This is to certify that the project work titled **STUDENT TEACHER ACADEMIC PORTAL**, submitted to the School of Computer Studies-MCA, RVS College of Arts and Science, in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications is a record of original project work done by **UDAYAKUMAR D** during the period Nov 2025 to Mar 2026 towards the partial fulfillment of the requirement for the award of MCA prescribed by RVS College of Arts and Science, Sulur.

Internal Supervisor

Declaration

DECLARATION

I am **UDAYAKUMAR D**, hereby declare that the project entitled **STUDENT TEACHER ACADEMIC PORTAL,** submitted to the School of Computer Studies, RVS College of Arts and Science, in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications is a record of original project work done by me during the period November 2025 to March 2026 under the internal supervision of **J . GNANA MANO SHEEBA, M.C.A., M.Phil., B.Ed., Assistant Professor, RVS College of Arts and Science (Autonomous)** From Bharathiar University, Coimbatore.

Signature of the Candidate

Acknowlegement

ACKNOWLEDGEMENTS

I express my sincere thanks to our Managing Trustee **Dr. K. Senthil Ganesh MBA (USA)., MS (UK)., Ph.D.,** for providing us with the adequate faculty and laboratory resources for completing my project successfully.

I express my sincere thanks to our Secretary Madam **Prof. Saramma Samuel.,** for providing us with the guidance for completing my project successfully.

I take this as a fine opportunity to express my sincere thanks to **Dr. T. Sivakumar M.Sc., M. Phil., Ph.D., Principal,** RVS College of Arts and Science (Autonomous) for giving me the opportunity to undertake this project.

I express my sincere thanks to **Dr. P. Navaneetham M.Sc., M.Phil., Ph.D., Director (Administration), Department of Computer Science** for the help and advice throughout the project.

I express my sincere thanks to **Dr. S. Yamini M.Sc., (CC)., M. Phil., Ph.D., Director (Academic), Department of Computer Science** for her valuable guidance and prompt correspondence throughout the curriculum to complete the project.

I express my gratitude to **J . GNANA MANO SHEEBA, M.C.A., M.Phil., B.Ed., Assistant Professor** for his valuable guidance, support, encouragement and motivation rendered by her throughout this project.

Finally, I express my sincere thanks to all other staff members and my dear friends, and all dear and near for helping me to complete this project.

UDAYAKUMAR D

Student Teacher Academic Portal
[STAP]

Abstract

ABSTRACT

Contents

CONTENTS

Certificate I

Declaration II

Acknowledgements III

List of Figures IV

List of Tables V

List of Abbreviations VI

Abstract VII

1. Introduction

1.1 An Overview of the Project 1

1.2 Mission of the Project 1

1.3 Background Study 1

1.3.1 A Study of the Existing System 1

2. System Analysis

2.1 A Study on the Proposed System 2

2.2 User Requirement Specification 2

2.2.1 Major Modules 2

2.2.2 Sub Modules 3

2.3 Software & Hardware Specification 4

2.3.1 Software Specification 4

MongoDB

Node.JS

React.JS

Express.JS

2.3.2 Hardware Specification 5

3. System Design and Implementation

3.1 Fundamentals of Design Concepts 6

3.1.1 Abstraction 6

3.1.2 Modularity 7

3.1.3 Refinements 7
3.2 Design Models 7
3.2.1 Structure Chart 8
3.2.2 System Flow Diagram 9
3.2.3 Data Flow Diagram 10
3.2.4 Software Engineering Model 12
3.2.5 Data Model Diagram 12
3.3 Design process
3.3.1 Software Architecture 14
3.3.2 Control Hierarchy 14
3.3.3 Structural Partitioning 14
3.3.4 Data structure 15
3.3.5 Software procedure 15
3.4 Data base Design 16
3.5 Input Design 16
3.6 Output Design 17

4. Testing and Implementation
4.1 Testing 18
4.1.1 white box testing and unit testing 18
4.2 Test case 19
4.3 System Maintenance 21

5. Conclusion
5.1 Directions for Future Enhancements 22

6. Bibliography
6.1 Web References 23
6.2 Book References 24

Annexure-A – Input Design 24
Annexure-B – Output Design 26
Annexure-C – Source Code 32

Introduction

1.INTRODUCTION

In educational institutions, effective communication and administrative transparency between students and teachers are vital for academic progress. Managing academic records such as attendance, scheduling face-to-face meetings, and conducting real-time consultations, however, is often challenging due to manual processes, mismatched availability, and the lack of proper communication channels. These challenges lead to wasted time, data inaccuracy, overlapping schedules, and disorganized management of academic engagement.

1.1 An overview of the project

The Student–Teacher Academic Portal (STAP) is a high-fidelity web-based application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It is designed to automate, streamline, and centralize the entire spectrum of student–teacher interactions, from attendance to virtual consultations.

The system allows students to:

- Track real-time attendance and academic progress across all subjects.
- Check teachers' real-time availability and request appointments easily.
- Engage in high-definition video consultations via integrated WebRTC.
- Receive instant notifications for academic updates and meeting confirmations.

Teachers can:

- Mark and manage student attendance with accurate digital logs.
- Handle appointment requests and manage their schedules efficiently.
- Conduct virtual office hours through a secure, institutional-grade calling platform.
- Maintain a centralized record of student engagement and communication history.

By providing a centralized and automated platform, the system improves communication, ensures data integrity, and saves time for students, teachers, and administrators. It enhances accessibility, transparency, and time management, resulting in an organized and efficient academic coordination process.

1.2 Mission of the Project

The mission of the Student–Teacher Academic Portal (STAP) is to provide an efficient, secure, and user-friendly digital platform that simplifies the process of academic management and meeting scheduling. The system aims to replace manual and unorganized methods with an automated solution that supports smooth and effective communication between students and faculty.

The key objectives of this project are to:

- Minimize manual effort in clinical attendance and record-keeping.
- Improve real-time communication between students and teachers.
- Eliminate scheduling conflicts by displaying real-time faculty availability.
- Enable high-quality virtual consultations using integrated WebRTC technology.
- Reduce administrative burden for faculty members through automated reporting.
- Provide transparency and traceability in attendance and appointment handling.
- Enhance academic support through timely and organized digital interactions.

Through these goals, the project seeks to create a reliable and modern academic management system suitable for today's educational institutions.

1.3 Background Study

Traditional systems for managing student–teacher interactions rely on manual communication methods such as paper-based attendance registers, waiting outside staff rooms, or passing messages through classmates.

These approaches often result in:

- Data redundancy and proxy attendance
- Miscommunication and missed appointments
- Inefficient time management
- Lack of a centralized record of academic progress and meetings

Many educational institutions still do not have a unified digital system that allows students to monitor their attendance, book academic consultations online, or receive real-time notifications. A well-designed portal like STAP can effectively address these issues and significantly enhance academic coordination and collaboration.

1.3.1 A Study of the Existing System

The existing administrative and scheduling system is predominantly manual and has several limitations:

- Attendance records are paper-based, leading to errors and delays in cumulative reporting.
- Students must personally approach teachers to request appointments, leading to inconsistent communication.
- Teachers manage appointments manually, which can result in overlapping schedules and confusion.
- There is no real-time visibility of teacher availability or student attendance status.
- Students often wait outside staff rooms for status checks, causing loss of time.
- Virtual communication depends on external, non-institutional links with no formal record.
- There is no centralized database to maintain comprehensive records of student–teacher engagement.

These manual processes reduce overall productivity and create a disorganized and inefficient workflow for the modern academic cycle.

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

System analysis is the process of studying the existing academic interactions and administrative workflows to identify their limitations and determining the requirements for the proposed system. The goal is to understand the current manual process of tracking attendance, scheduling face-to-face meetings, and conducting consultations, while identifying problems such as communication delays, data inaccuracy, and inefficient record-keeping. The proposed Student–Teacher Academic Portal (STAP) is an automated solution designed to overcome these challenges.

The proposed system is designed to streamline academic interactions by providing a centralized platform. It allows students to monitor their progress, teachers to manage their academic duties including attendance and video calling, and administrators to oversee the repository of departments and subjects. This integration ensures effective communication, reduces administrative friction, and improves overall productivity for the institution.

2.1 A Study on the Proposed System

The proposed system is a high-fidelity web-based platform that automates the academic and communication processes between students and teachers. It is designed to provide efficient management of attendance, appointments, and virtual consultations, eliminating the limitations of manual scheduling.

The system allows:

- Students to check real-time attendance status, faculty availability, and participate in virtual office hours.
- Teachers to update digital attendance, manage meeting schedules, and host secure video consultations without conflicts.
- Administrators to maintain registry records, manage user accounts, and monitor portal activities centrally.

This system enhances institutional productivity, transparency, and coordination among all users while ensuring a structured and reliable academic management process through the MERN stack.

2.2 User Requirement Specification

The User Requirement Specification (URS) defines the functional and non-functional expectations of the portal from the perspective of its users. It ensures that the system meets the complex needs of students, teachers, and administrators effectively.

Functional Requirements:

- Students should be able to view attendance summaries, book available slots, and join video consultations via WebRTC.
- Teachers should be able to mark digital attendance registers, respond to appointment requests, and host virtual meetings.
- Administrators should be able to manage the global registry of Departments and Subjects and maintain user data.
- The system should provide real-time updates through Socket.io for notifications and chat messages.

Non-Functional Requirements:

- The system should feature a "Sophisticated Pro" glassmorphism UI that is user-friendly and intuitive.
- It should be responsive and accessible via standard web browsers across desktop and mobile devices.
- The system must ensure high data security using JWT authentication and password encryption.
- Performance should be optimized to handle real-time video signaling and concurrent user sessions.

2.2.1 Major Modules

The Student–Teacher Academic Portal (STAP) is divided into three major modules to manage the roles and functionalities of different users efficiently:

1. Student Module

Allows students to view their attendance metrics, request academic appointments, and engage in real-time chat or video calls with teachers.

2.Teacher Module

Enables teachers to manage digital attendance registers, approve or reject meeting requests, and host virtual consultation rooms.

3.Administrator Module

Provides administrators with the ability to manage the institutional registry (Departments and Subjects), oversee user accounts, and maintain system integrity.

2.2.2 Sub Modules

Each major module of the system is divided into sub-modules to manage specific tasks efficiently and ensure that users can perform their functions seamlessly.

1.Student Sub-Modules:

- Attendance Tracker: Students can access real-time status of their presence in each subject to maintain academic eligibility.
- Appointment Request: Students can submit meeting requests by selecting convenient slots based on the teacher's live schedule.
- Virtual Office Join: A dedicated sub-module for joining high-definition video consultations via WebRTC with a single click.

2.Teacher Sub-Modules:

- Attendance Manager: Teachers can toggle student presence in an interactive table, with records instantly synced to the database.

- Meeting Responders: Teachers can review incoming requests and manage their weekly schedule to avoid appointment overlaps.
- Consultation Host: An integrated platform where teachers can conduct secure face-to-face video calls and chat with students.

3.Administrator Sub-Modules:

- Registry Control: Administrators can add, update, or remove Departments and Subjects that form the backbone of the institution.
- User Accountability: Managing student and teacher credentials and permissions to ensure only authorized access.
- Portal Monitoring: Tracking notification logs and appointment trends to maintain an organized institutional workflow.

2.3 Software requirement specification

System Overview:

The Student–Teacher Academic Portal (STAP) is a web-based integrated system built on the MERN stack to automate academic administration. It provides a unified ecosystem where attendance is digitized, appointments are scheduled automatically, and consultations are conducted virtually. The system eliminates the "data void" found in manual processes and ensures a high level of transparency for both faculty and students.

Problem Statement:

In many educational institutions, attendance tracking and meeting scheduling are handled manually through paper registers or separate apps, leading to proxy attendance, miscommunication, and loss of records. Students often lack real-time visibility into their attendance or teacher availability, while faculty face high administrative burdens. A centralized, MERN-based system is required to automate these workflows, provide real-time status updates, and facilitate secure virtual communication within a single platform.

2.4 System Specification

The system specification describes the technical components required for the development and deployment of STAP. It focuses on the MERN stack architecture and real-time protocols like WebRTC and Socket.io to ensure smooth operation.

2.4.1 React.js

React.js is used as the frontend framework for developing the "Sophisticated Pro" interface. It enables the creation of dynamic, responsive, and interactive pages for students and teachers. Its component-based architecture ensures that features like the Attendance Table and Video Call UI are modular, reusable, and handle state updates seamlessly through Redux.

2.4.2 Node.js & Express.js

Node.js serves as the server-side runtime, enabling high-performance handling of concurrent user requests. Together with Express.js, it manages the portal's API, facilitating secure authentication (JWT), data processing for attendance, and real-time signaling for WebRTC video calls.

2.4.3 Hardware configuration

- Processor: Intel i3 (12th Gen) or higher
- RAM: Minimum 8 GB
- Storage: 250 GB HDD/SSD or above
- Network: High-speed internet connection (recommended for video consultations)

2.4.4 Software configuration

- Operating System: Windows, Linux, or macOS
- Frontend: React.js with Redux Toolkit
- Backend: Node.js with Express.js
- Database: MongoDB (Document-oriented NoSQL)
- Communication: Socket.io and WebRTC
- Development Tools: VS Code and Vite

Design Modules

3. SYSTEM DESIGN AND IMPLEMENTATION

System design is the technical solution for the creation of the Student–Teacher Academic Portal (STAP). This phase focuses on the detailed implementation of the requirements identified during the analysis phase. It emphasizes translating design specifications into a functional performance specification. The system design for STAP is divided into two primary phases:

Logical Design:

Defines the academic data flow and system processes, such as resource distribution and messenger communication, without specifying physical implementation.

Physical Design:

Converts the logical design into technical specifications including MongoDB schemas, WebRTC signaling logic, and server-side Socket.io handlers.

3.1 Fundamentals of Design Concepts

The design of the STAP web application involves several critical components focused on seamless interaction between students and faculty. Key objectives include:

- Creating an intuitive interface for students to access attendance reports and uploaded academic resources.
- Implementing a real-time messenger and notification engine using Socket.io for instant feedback.
- Designing a responsive dashboard for teachers to manage attendance, resource uploads, and appointment requests.
- Ensuring secure and fast data transfer between the React-Redux frontend and the Node.js backend using JWT.

3.1.1 Abstraction

In STAP, abstraction is applied by modeling complex institutional entities like 'Academic Resources' and 'Real-time Messages' into manageable software objects. For instance, the system treats an 'Appointment' as an abstract entity with properties like time, status, and participants, hiding the underlying complexity of WebRTC signaling and database indexing. This allows the development team to focus on the educational workflow rather than low-level data management.

3.1.2 Refinement

Refinement in STAP follows a top-down approach. The high-level function 'Academic Consultation' is refined into smaller, executable steps: User Authentication, Resource Access, Appointment Selection, and Live Video Joining. This progressive detail ensures that every edge case, such as a student attempting to join a call without a confirmed slot, is addressed during implementation.

3.1.3 Modularity

STAP is designed as a modular system to ensure scalability and ease of maintenance. The application is divided into independent modules: Authentication (handling permissions), Resource Module (processing video/file uploads), Messenger Module (enabling real-time chat), and Attendance Manager (tracking student presence). This modularity allows for high cohesion within modules and low coupling between them.

3.2 Design Notations

3.2.1 System Structure Chart

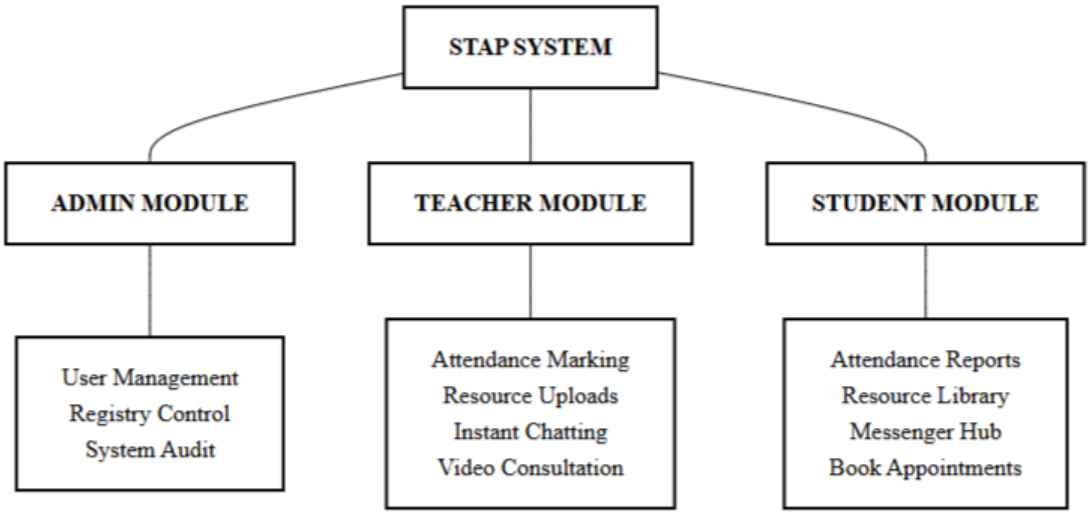


Figure 3.1 - System structure chart

3.2.2 System Flow Diagram

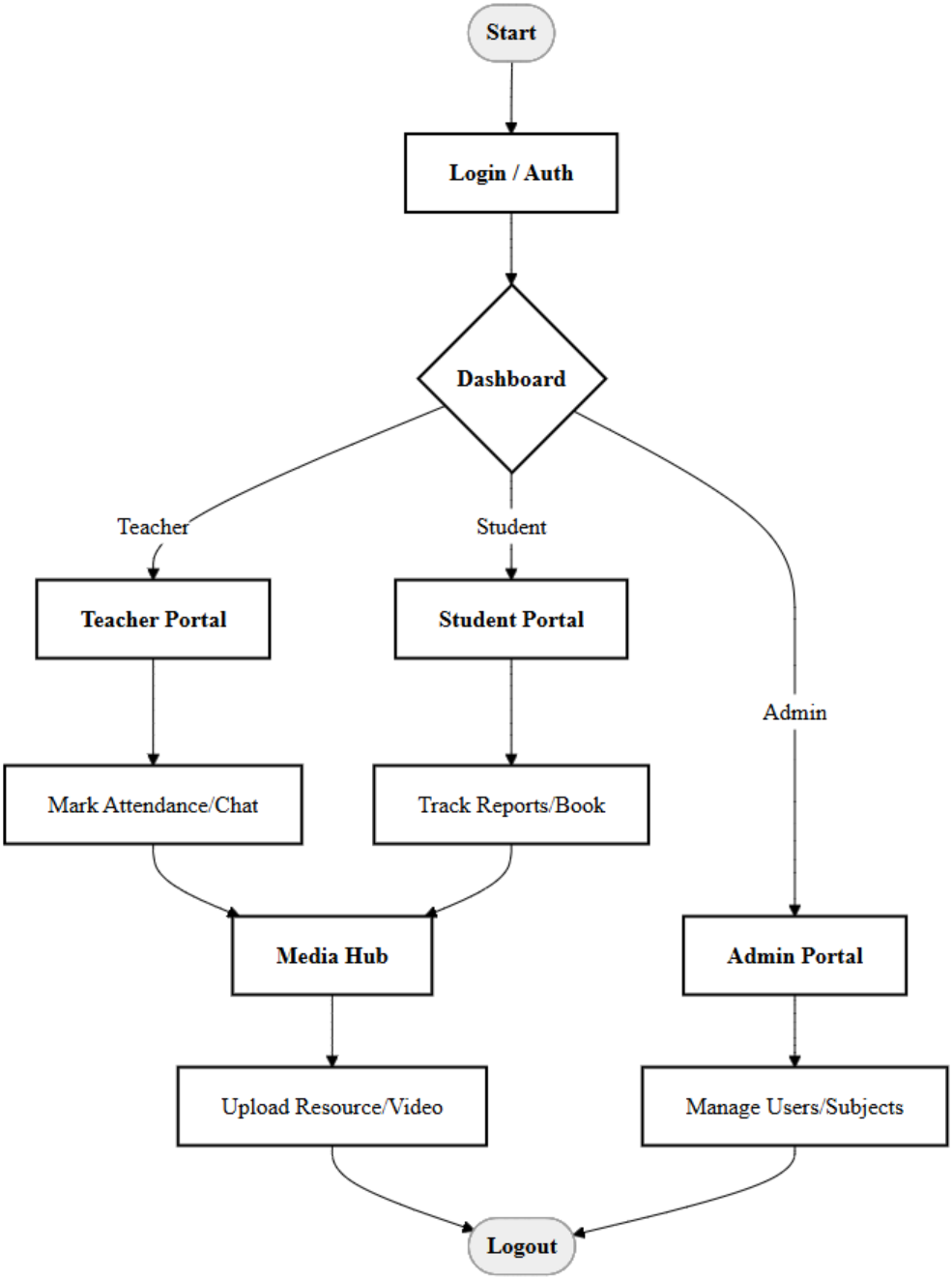


Figure 3.2 - System flow diagram

Illustrates the end-to-end operational logic from login to the completion of academic tasks.

3.2.3 Data Flow Diagram / UML

A Data Flow Diagram (DFD) for STAP describes the dynamic movement of academic data between users and the database. It clarifies the transformations involved, such as converting a teacher's resource upload into a student's accessible lecture link.

EXTERNAL ENTITY

Key entities in STAP include the Student, Teacher, and System Administrator.

PROCESS

Processes include 'Mark Attendance', 'Upload Resource', 'Sync Message', and 'Validate Appointment'.

DATAFLOW

Data flows such as 'Resource Metadata', 'Real-time Chat Packet', and 'Attendance Log' represent the movement of information across the system.

LOGIN DFD:

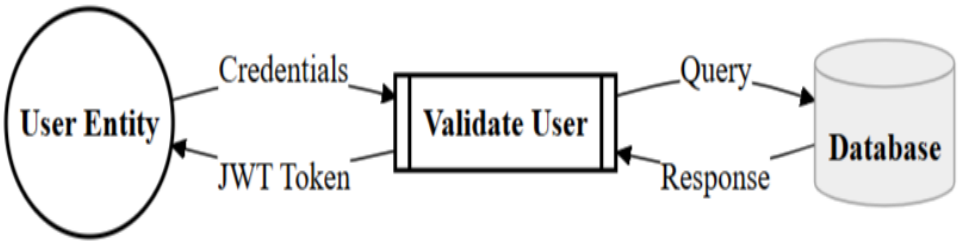


Figure 3.3

Shows user authenticating against the MongoDB User Collection using JWT.

REGISTRATION DFD:

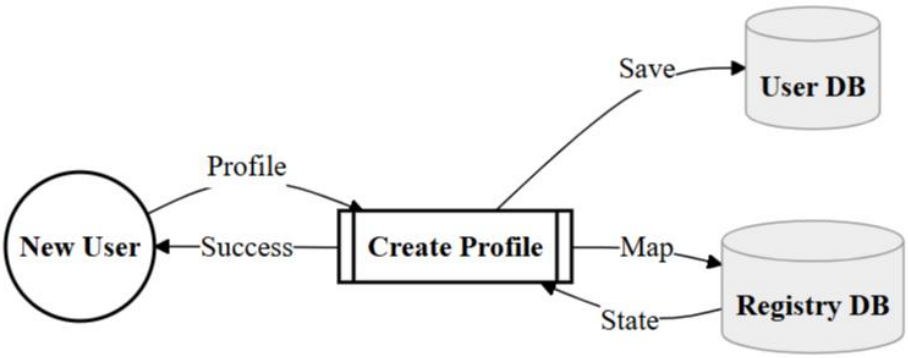


Figure 3.4

Shows new student/teacher profile creation and department mapping.

ADMIN DFD:

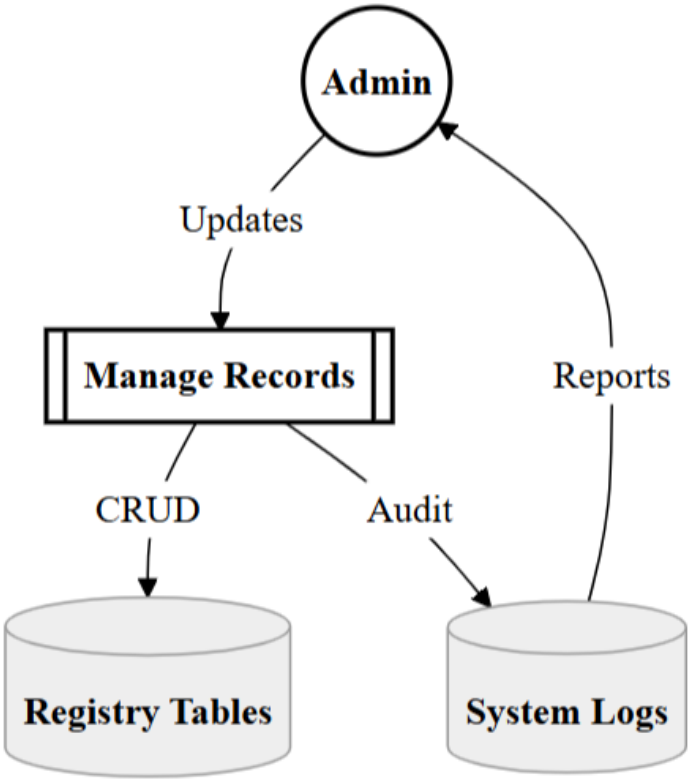
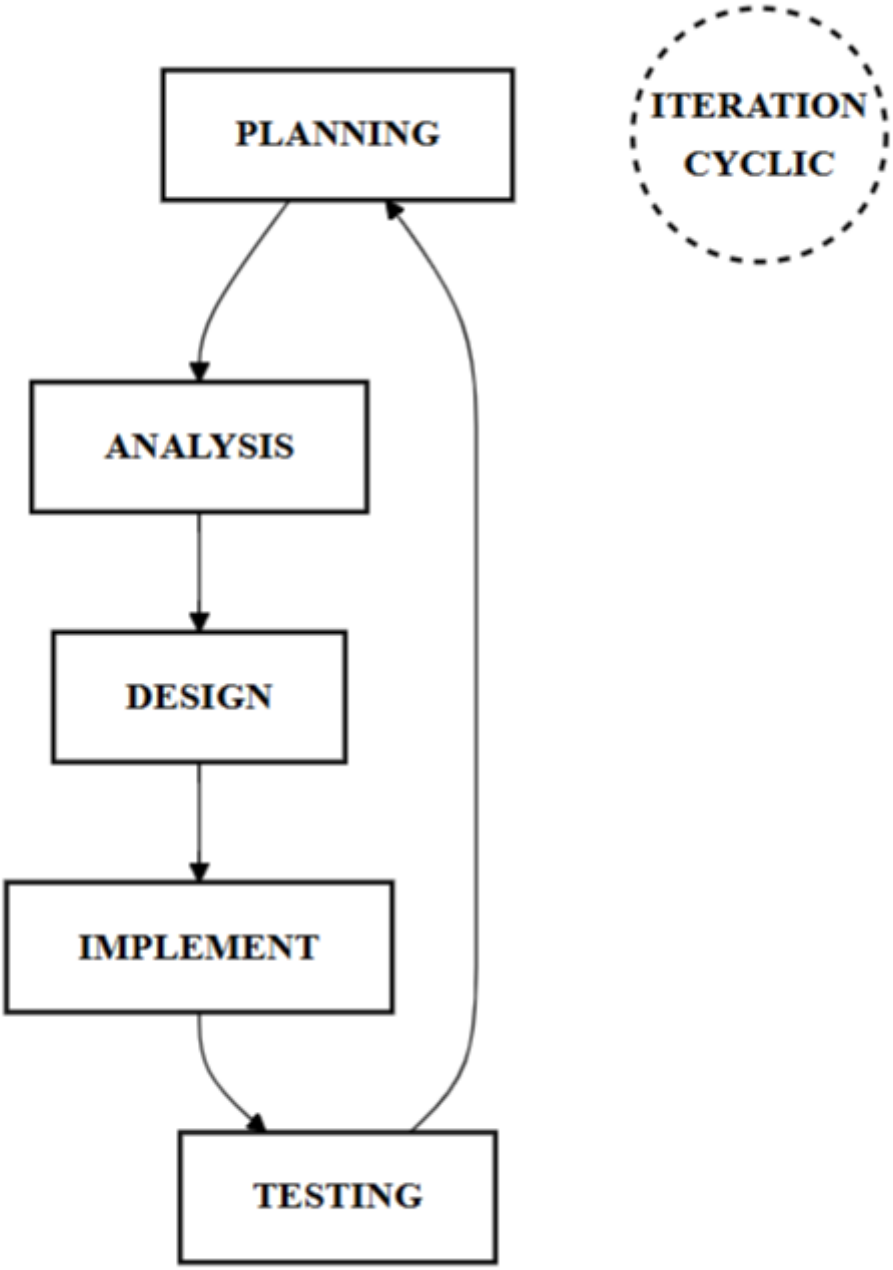


Figure 3.5

Shows administrative control over users, departments, and system-wide reports.

3.2.4 Software Engineering Model

STAP utilizes an Iterative and Incremental development model. This allows for core attendance and registry features to be built first, followed by high-definition communication enhancements like WebRTC video calling. User feedback is used to refine the "Sophisticated Pro" glassmorphism interface in subsequent sprints.



3.3 Design Process

3.3.1 Software Architecture

The software architecture of STAP is based on the MERN stack (MongoDB, Express, React, Node.js). This Model-View-Controller (MVC) approach ensures a clear separation of concerns, with React handling the View, Express/Node managing the Controller logic, and MongoDB acting as the Model for data persistence.

3.3.2 Control Hierarchy

The control hierarchy defines the operational layers of STAP. The Top Layer (Portal UI) interacts with the Middle Layer (REST API & WebSockets), which in turn communicates with the Bottom Layer (Data Store). This ensures that permissions and data integrity are enforced at every level.

3.3.3 Structural Partitioning

STAP is partitioned both vertically (separating Student, Teacher, and Admin flows) and horizontally (separating Attendance inputs, Resource processing logic, and Messenger events). This partitioning improves system reliability and simplifies unit testing.

3.3.4 Data Structure

Data structures are chosen for maximum efficiency. Attendance logs are stored in indexed collections, while Messenger history uses time-sorted schemas. User sessions are managed using JSON Web Tokens (JWT) for secure, stateless communication.

3.4 Table Design

The database design includes five main collections: User (roles and auth), Attendance (presence logs), Resource (video/file metadata), Appointment (booking details), and Subject (registry info). Each table is normalized to reduce data redundancy.

3.5 Input Design

Input design for STAP focuses on minimizing entry errors. Attendance registers use toggle-based forms, while Resource uploads feature drag-and-drop interfaces with type validation. Frontend forms ensure all required fields are filled before submitting to the server.

3.6 Output Design

Output in STAP includes the Attendance Summary Report, Resource Library, and Live Consultation Interface. Data is presented through clean, responsive tables and high-definition video frames with status indicators (e.g., Confirmed, Uploaded, Live).

3.7 Development Approach

An Agile development approach is adopted, utilizing Git for version control and iterative testing. Each feature of STAP (Attendance, Resource Upload, Messenger) undergoes separate unit testing before integration, ensuring a robust and professional academic experience.

Testing and Implementation

4. TESTING AND IMPLEMENTATION

4.1 Testing

Software testing is a critical process aimed at identifying errors and defects in software. It serves as a final review of all the previous phases of software development, including specification, design, and code generation. For the Student Teacher Academic Portal (STAP), testing ensures that real-time features like video calls and attendance tracking function reliably under various conditions.

4.1.1 System Testing

System testing of STAP involves the integrated testing of the entire software to verify that it meets the specified requirements. This includes testing the end-to-end flow from user registration to virtual consultation. It ensures that the React frontend, Node.js backend, and MongoDB database work together seamlessly.

4.1.2 White Box Testing:

In white box testing, knowing the internal working of the product, tests can be conducted to ensure that internal operations are performed according to specification and all internal components have been adequately exercised. In STAP, white box testing is used to verify the logic of the WebRTC signaling flow and Socket.io event emissions.

4.1.3 Unit Testing :

Unit testing emphasizes the verification effort on the smallest unit of software design—a software component or module. In STAP, units such as the "Attendance Marking Component," "Message Sender Controller," and "Appointment Validator" are tested independently using JEST to catch bugs early in the development lifecycle.

```
PASS  tests/mongoConnection.test.js
  √ MongoDB connection (690 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.471 s, estimated 3 s
Ran all test suites matching tests/mongoConnection.test.js.
Force exiting Jest: Have you considered using `--detectOpenHandles` to detect asy
nc operations that kept running after all tests finished?

E:\SEM-4\STABS-BACKEND>
```

Figure 4.1

Unit Testing - MongoDB Connection Pass Result

4.1.4 API Testing Results (Postman)

API testing is conducted to ensure that the backend endpoints are functioning correctly and that data is being stored and retrieved from the cloud database as expected.

POST : https://rvscas-backend.onrender.com/api/auth/register

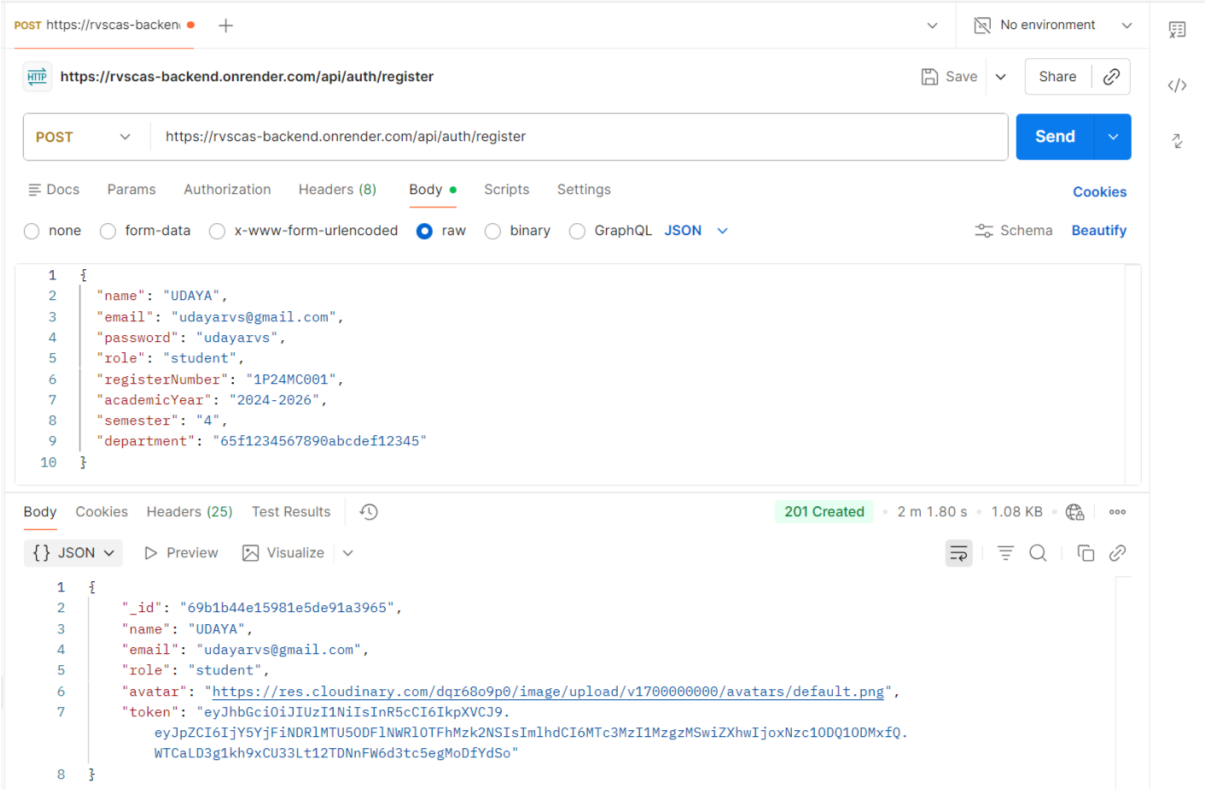


Figure 4.2

API Testing - Successful User Registration (201 Created)

POST : https://rvscas-backend.onrender.com/api/auth/login

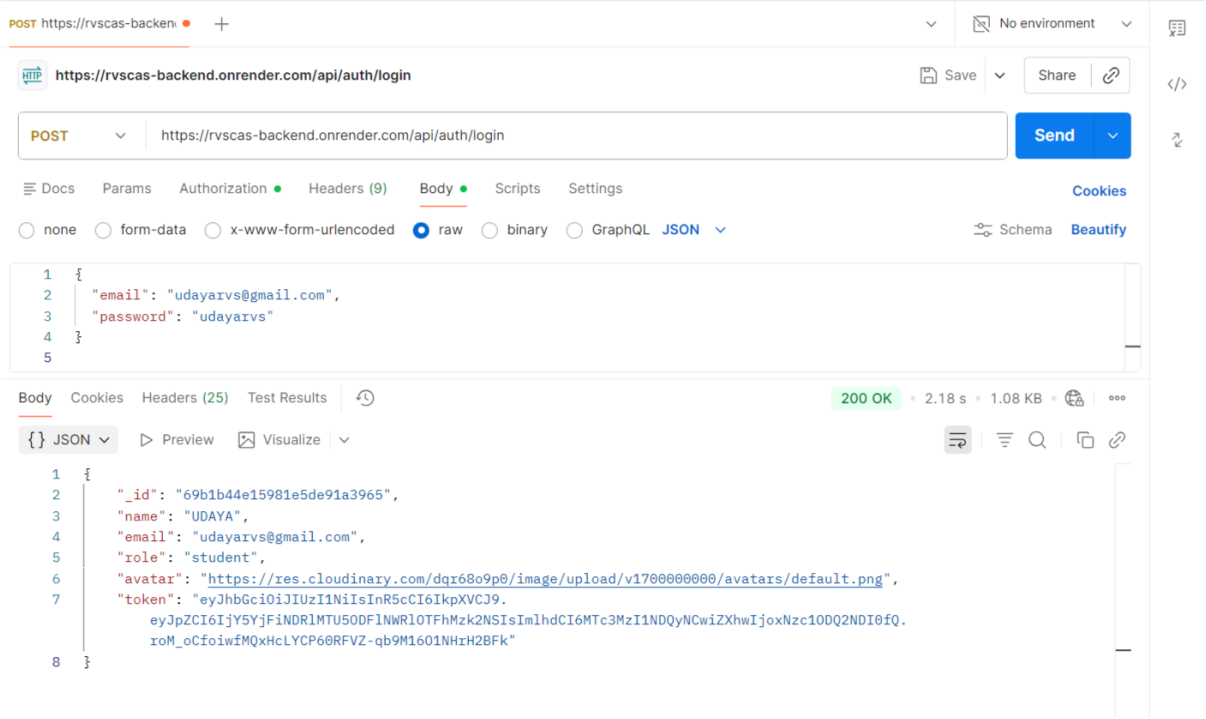


Figure 4.3

API Testing - User Authentication and JWT Token Generation

GET : https://rvscas-backend.onrender.com/api/auth/profile

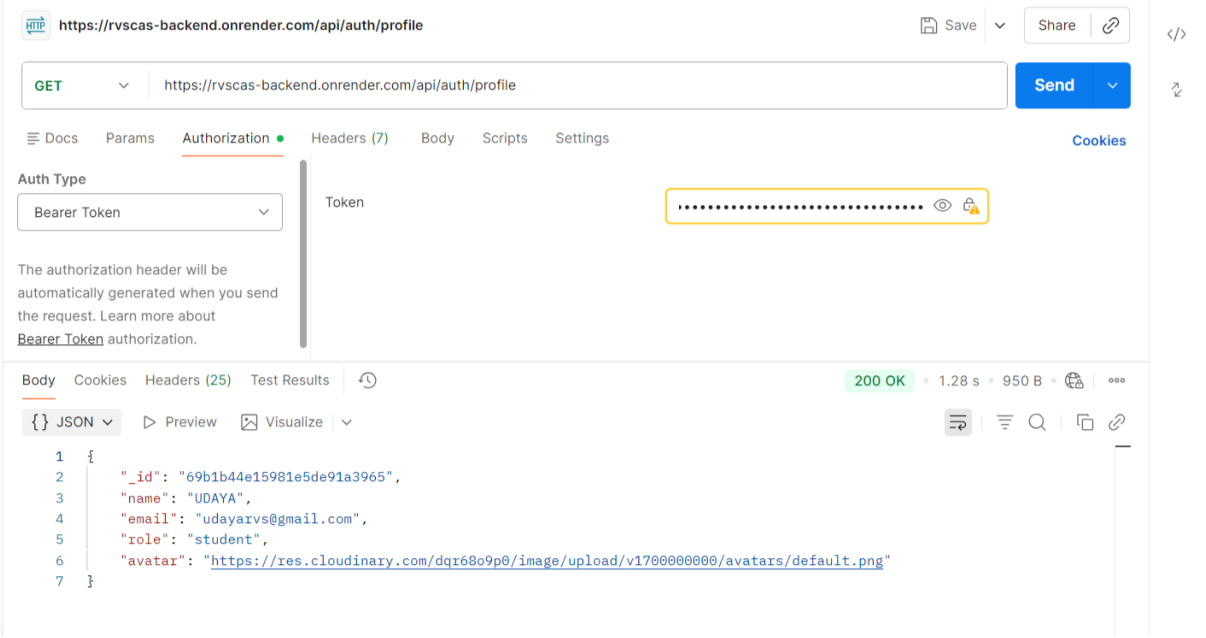


Figure 4.4

API Testing - Retrieval of User Profile Data from Backend

4.2 System Implementation

System implementation is the stage of the project where the theoretical design is turned into a practical system. For STAP, this involved setting up the MERN production environment on Render.com and configuring the MongoDB Atlas cloud database.

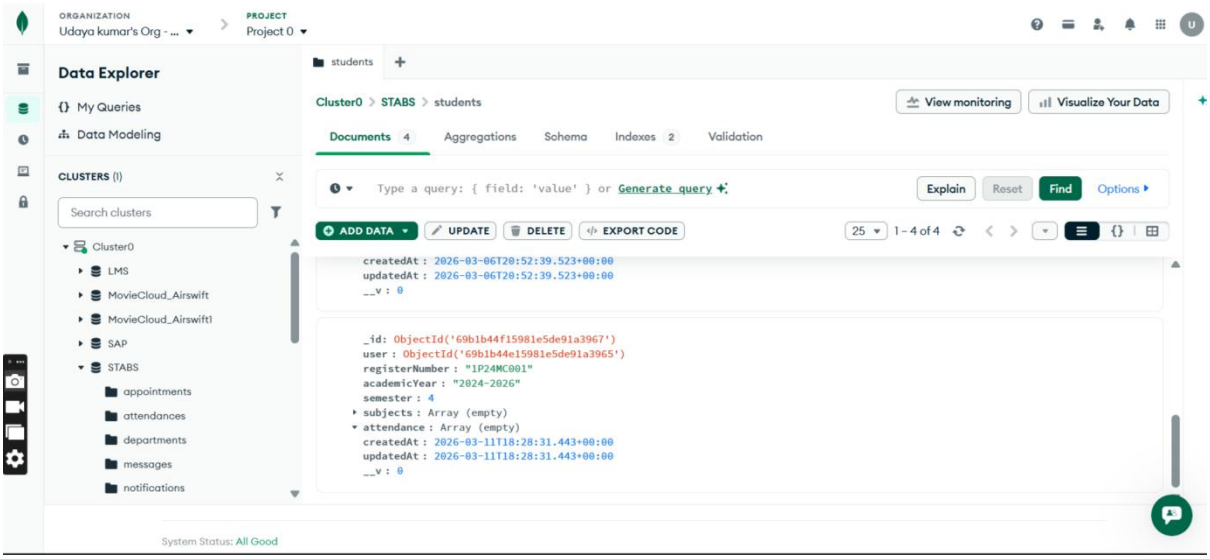


Figure 4.5

Cloud Database Implementation - MongoDB Atlas Collection View

4.2.1 System Maintenance

Maintaining a MERN (MongoDB, Express.js, React.js, Node.js) stack in STAP involves several key activities:

- Regularly updating dependencies: Keeping npm packages updated to patch security vulnerabilities.

- Monitoring server performance: Using Node.js monitoring tools to ensure low latency.

- Backing up data: Regularly backing up MongoDB Atlas clusters and Cloudinary media.

- Log Management: Reviewing system logs to identify failed login attempts or broken API requests.

Conclusion

5. CONCLUSION

In summary, developing the Student Teacher Academic Portal (STAP) using the React.js, Node.js, and MongoDB (MERN) stack establishes an agile and highly effective platform for linking prospective students with their academic faculty and course coordinators. This technology combination offers unparalleled versatility across platforms, real-time functionality through WebSockets, and the necessary scalability to manage vast amounts of academic-related data. The implementation of this system not only guarantees a seamless and user-friendly experience for both students and teachers but also champions the cause of academic welfare by ensuring that educational resources and consultations are accessible regardless of physical boundaries.

By harnessing the collective strengths of React.js for the dynamic frontend and Node.js with Express for the robust backend, we have established a highly efficient, user-centric environment. The utilization of MongoDB for data storage ensures the adaptability required for managing complex information such as attendance records, video lecture metadata, and student registries. To successfully execute this project, we adhered to industry best practices, including the definition of clear project objectives, the creation of a meticulously planned modular structure, the design of essential interactive features like WebRTC-based video calling, and the implementation of comprehensive system-wide testing.

5.1 Directions for Future Enhancements

Future enhancements for the Student Teacher Academic Portal (STAP) should focus on further improving the user experience and expanding the system's utility within the global academic ecosystem. The following high-fidelity features are proposed for future development:

1.AI-Powered Personalized Learning & Predictive Analytics:

Implementing advanced machine learning algorithms that analyze individual student learning patterns, assessment scores, and engagement metrics. This would enable the system to provide personalized course recommendations and predictive alerts for faculty, identifying students who may require additional academic support before their performance declines.

2.Comprehensive Gamification & Engagement Engine:

Introducing a sophisticated reward-based system featuring digital badges, achievement tiers, and progress leaderboards. By gamifying tasks such as consistent attendance, timely resource completion, and active participation in discussion forums, the portal can significantly increase intrinsic motivation and student engagement.

3.Native Cross-Platform Mobile Application:

Beyond a responsive web interface, developing dedicated native applications for Android and iOS using frameworks like React Native. This will provide users with seamless access to on-the-go features, including offline resource viewing, push notifications for instant class updates, and biometrically secured logins.

4.Advanced AI Academic Chatbot & Virtual Assistant:

Integrating a Natural Language Processing (NLP) based chatbot capable of handling complex student queries. This virtual assistant would provide 24/7 support for timetable management, attendance inquiries, and administrative FAQs, while also acting as a first-line support for technical troubleshooting within the portal.

5.Integrated Online Examination & Secure Proctoring Module:

Expanding the platform into a full-scale digital campus by including a secure examination portal. This module would feature AI-based automated proctoring (using facial recognition and screen monitoring) to ensure the integrity of remote assessments, along with automated grading for objective-based exams.

6.Virtual Reality (VR) Classroom Integration:

Exploring the potential of WebVR or Virtual Reality to host immersive 3D classroom environments. This would allow students to participate in specialized laboratory simulations or interactive lectures that transcend the limitations of traditional 2D video conferencing.

7.Blockchain-Based Certification & Credentialing:

Implementing a decentralized ledger system to issue digital certificates and academic transcripts. This ensures that student credentials remain tamper-proof, easily verifiable by third-party organizations, and securely managed within the student’s digital profile.

Bibliography

BIBLIOGRAPHY

Web References:

React JS

https://react.dev/

https://react.dev/learn

https://www.tutorialspoint.com/reactjs/index.htm

https://reactnavigation.org/

Node.js

https://nodejs.org/en/learn

https://nodejs.org/en/docs/

https://www.tutorialspoint.com/nodejs/index.htm

MongoDB

https://www.mongodb.com/docs/

https://www.tutorialspoint.com/mongodb/index.htm

https://www.mongodb.com/atlas/database

Express.js

https://expressjs.com/

https://www.tutorialspoint.com/expressjs/index.htm

https://expressjs.com/en/starter/installing.html

Other Essential Tools

https://www.npmjs.com/

https://socket.io/docs/

https://webrtc.org/

https://github.com/nvm-sh/nvm

Book References:

1. Learning React:

A Hands-On Guide to Building Web Applications Using React and Redux

Author: Kirupa Chinnathambi

Publisher: Addison-Wesley Professional

Year: 2018

Edition: 2nd Edition

Description:

This book provides a solid foundation in React and Redux. It explains how to create reusable UI components, manage application state, and build scalable single-page applications, which are core technologies used in the Student Teacher Academic Portal (STAP).

Link:

https://www.pearson.com/store/p/learning-react/P200000000179

2. Pro MERN Stack:

Full Stack Web App Development with Mongo, Express, React, and Node

Author: Vasan Subramanian

Publisher: Apress

Year: 2019

Edition: 2nd Edition

Description:

This book explains how to develop full-stack web applications using the MERN stack. It covers MongoDB database design, Express and Node backend development, and React frontend integration.

Link:

https://link.springer.com/book/10.1007/978-1-4842-4391-6

3. Full-Stack React Projects:

Learn MERN Stack Development by Building Modern Web Apps

Author: Shama Hoque
Publisher: Packt Publishing
Year: 2020
Edition: 2nd Edition

Description:

Provides hands-on MERN projects demonstrating how to build real-time applications using modern technologies such as Socket.io and REST APIs.

Link:

https://www.packtpub.com/en-us/product/full-stack-react-projects-9781788835534

4. The Road to React

Author: Robin Wieruch
Publisher: Independently Published
Year: 2020
Edition: 4th Edition

Description:

Focuses on React fundamentals, clean coding practices, and scalable frontend architecture for building professional React applications.

Link:

https://www.roadtoreact.com/

5. MongoDB: The Definitive Guide

Authors: Shannon Bradshaw, Eoin Brazil, Kristina Chodorow

Publisher: O'Reilly Media

Year: 2019

Edition: 3rd Edition

Description:

Explains MongoDB architecture, schema design, indexing, and performance optimization. It is useful for designing scalable databases for student and teacher profiles.

Link:

https://www.oreilly.com/library/view/mongodb-the-definitive/9781491954454/

6. Real-Time Communication with WebRTC

Authors: Salvatore Loreto, Simon Pietro Romano

Publisher: O'Reilly Media

Year: 2014

Description:

Describes WebRTC technology used for implementing browser-based video, voice, and data communication in real-time web applications.

Link:

https://www.oreilly.com/library/view/real-time-communication-with/9781449371869

7. The Definitive Guide to HTML5 WebSocket

Authors: Vanessa Wang, Frank Salim, Peter Moskovits

Publisher: Apress

Year: 2013

Description:

Explains WebSocket technology that enables real-time communication between client and server, useful for instant messaging systems and live updates.

Link:

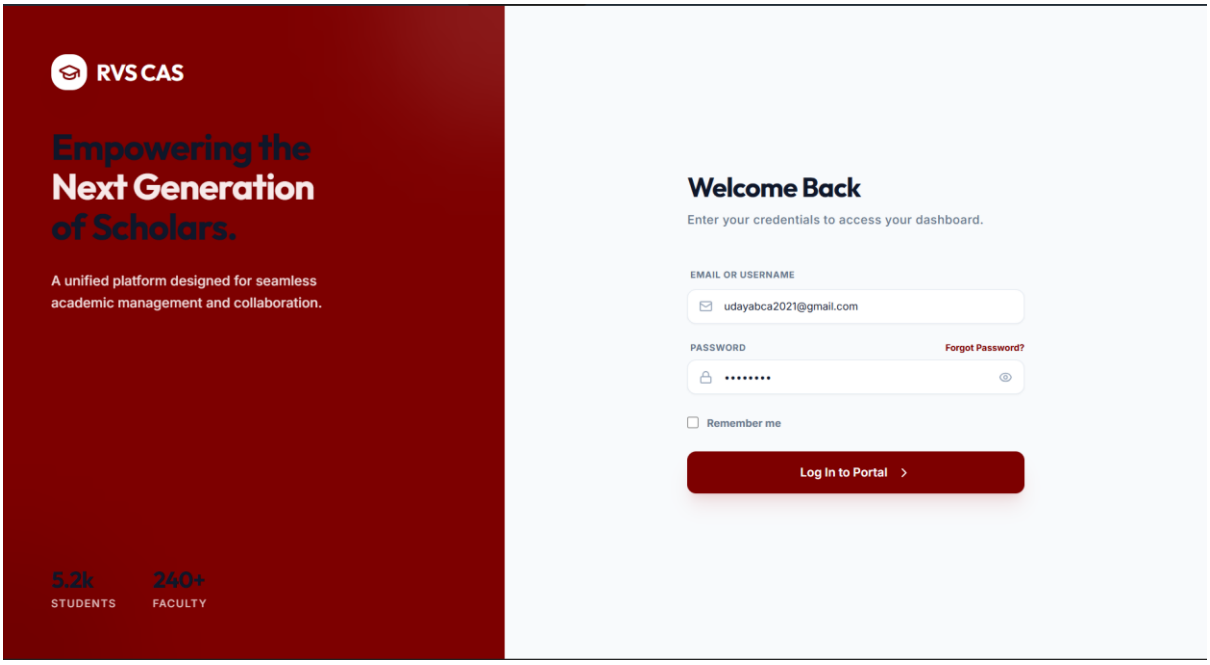
https://link.springer.com/book/10.1007/978-1-4302-4741-8

These books provide in-depth knowledge on MERN stack development, from React front-end programming to server-side scripting with Node.js and managing databases using MongoDB. They also offer insights into implementing real-time communication features relevant to the STAP system.

Annexures

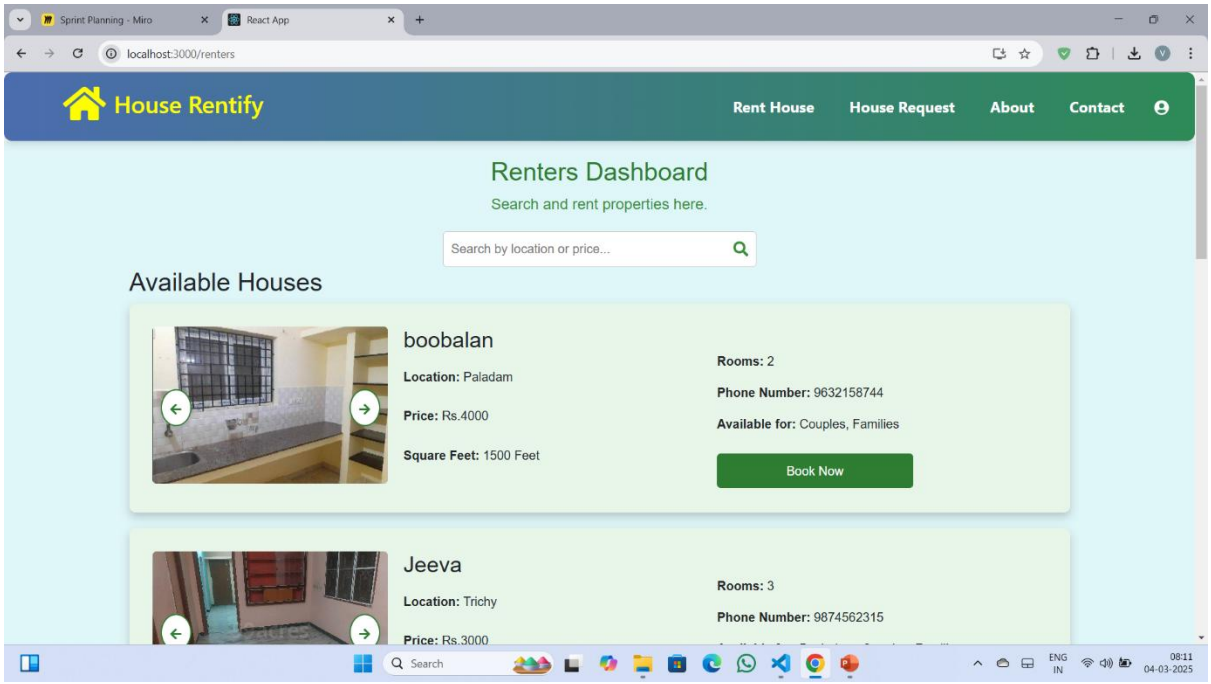
Annexure-A – Input Design

Login Screen

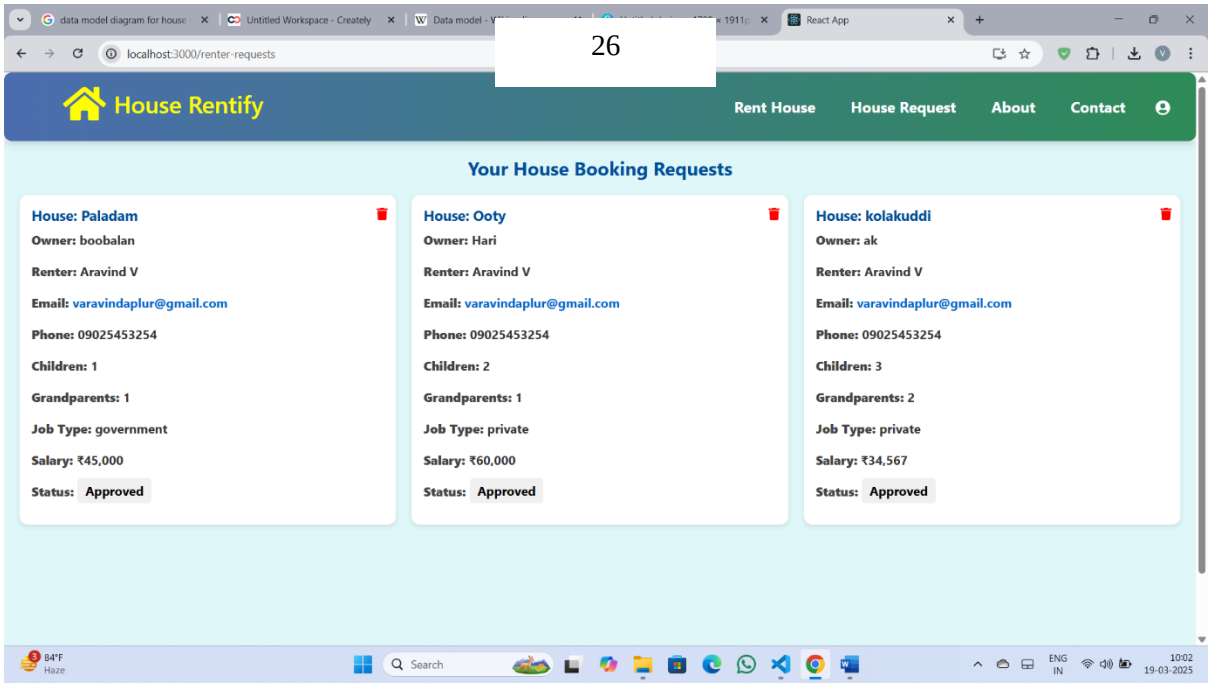


Annexure-B – Output Design

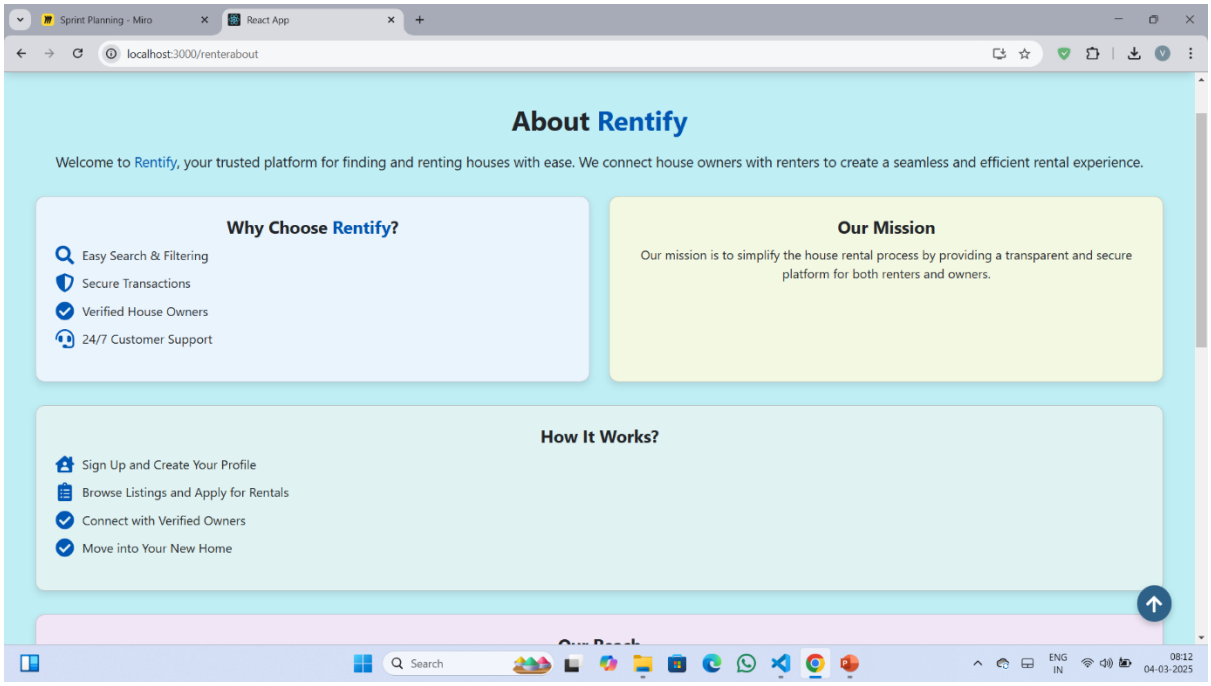
Renter Home Screen



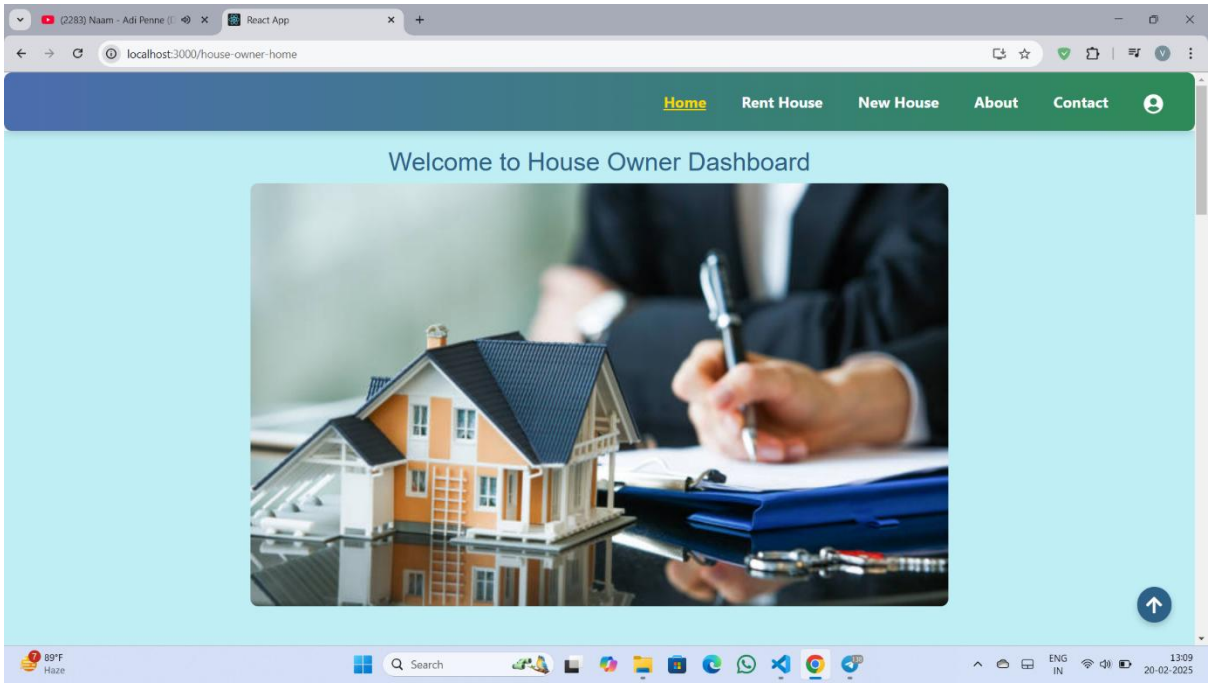
Renter House Request Screen



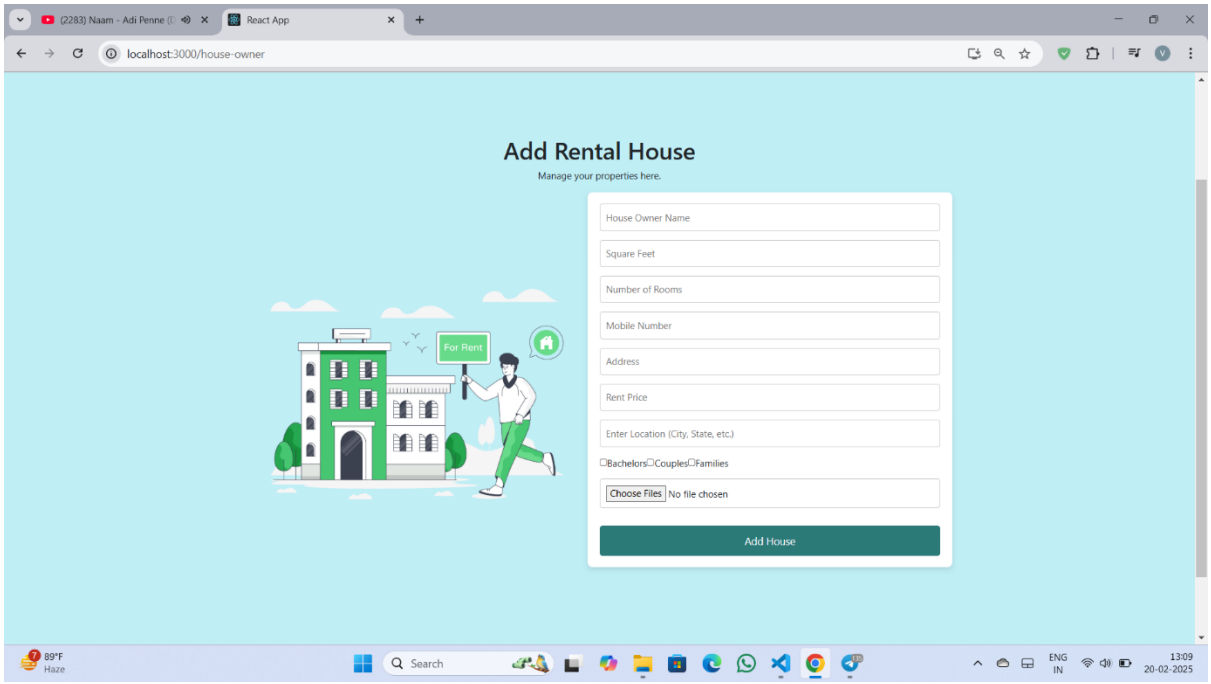
Renter About Screen



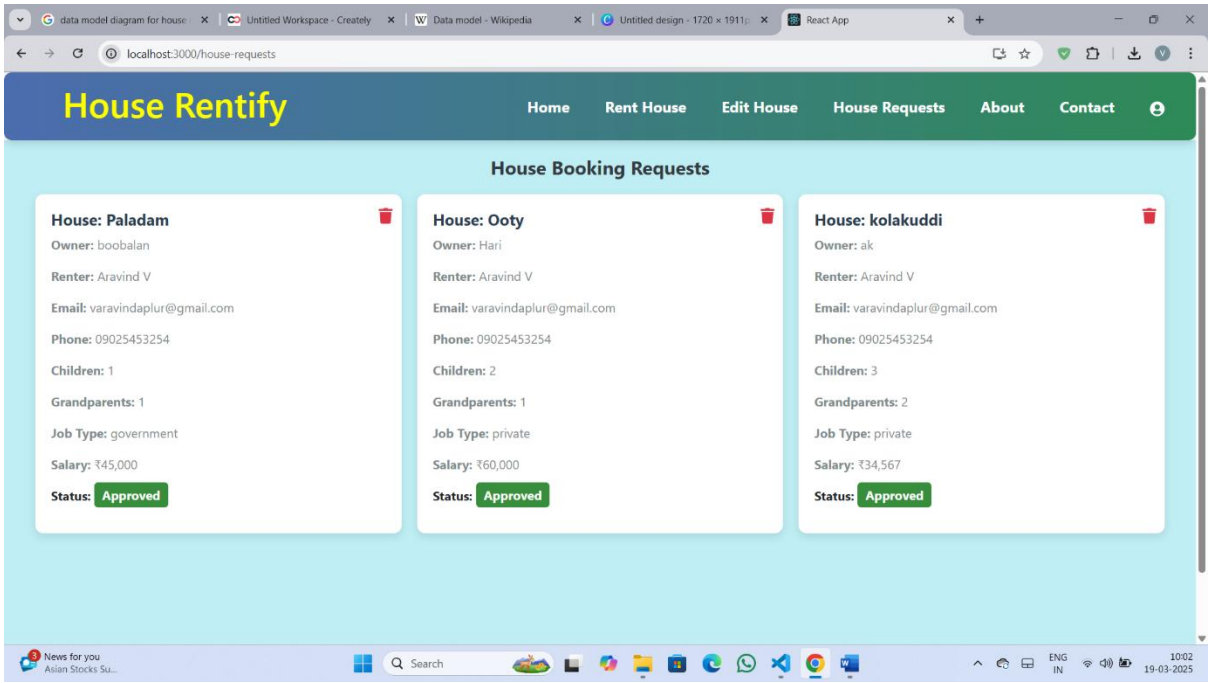
House Owner Home Screen



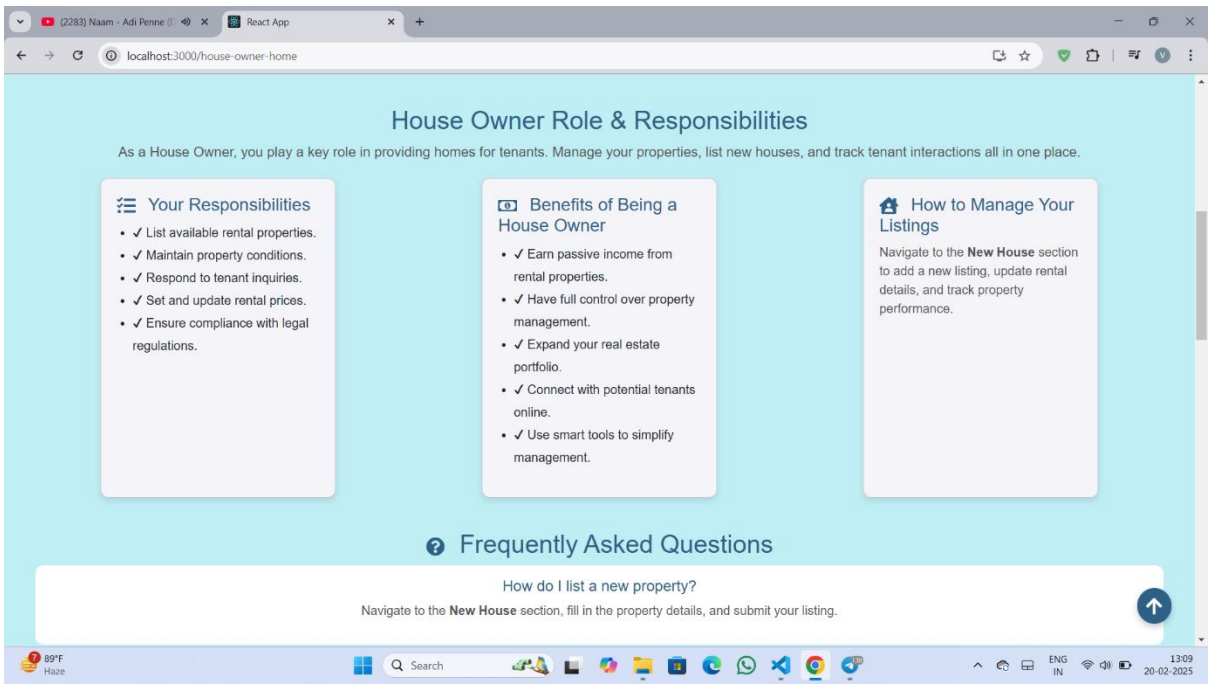
House Owner Add House Screen



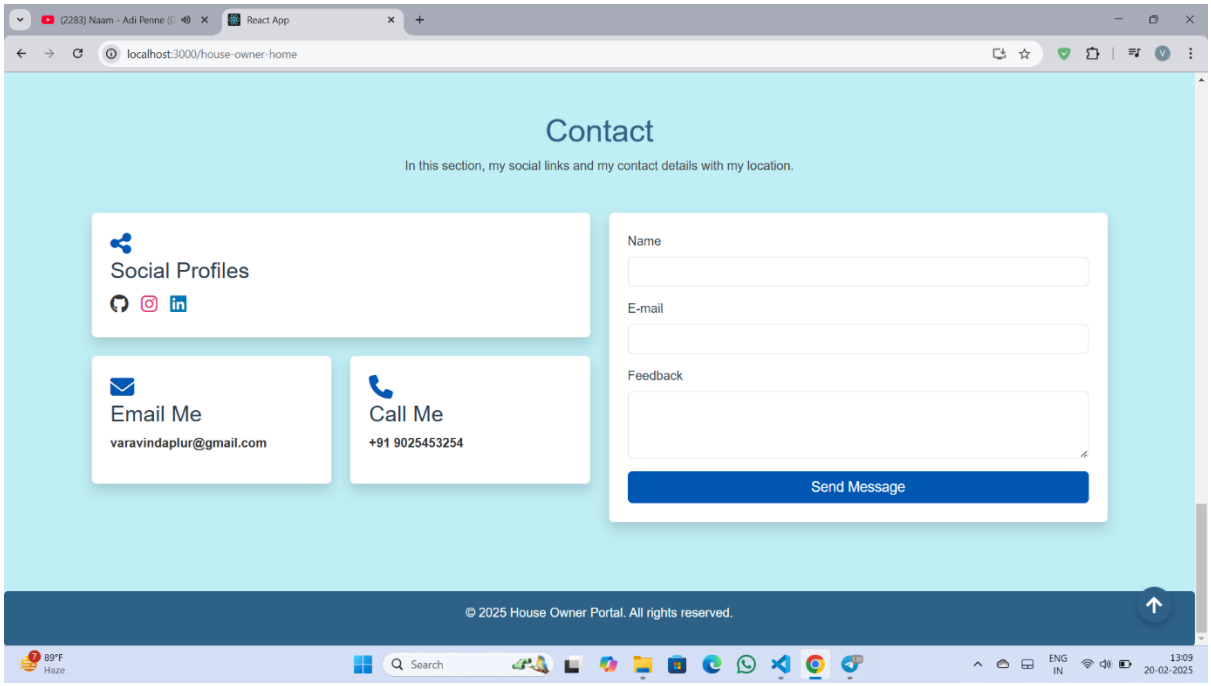
House Owner Request Screen



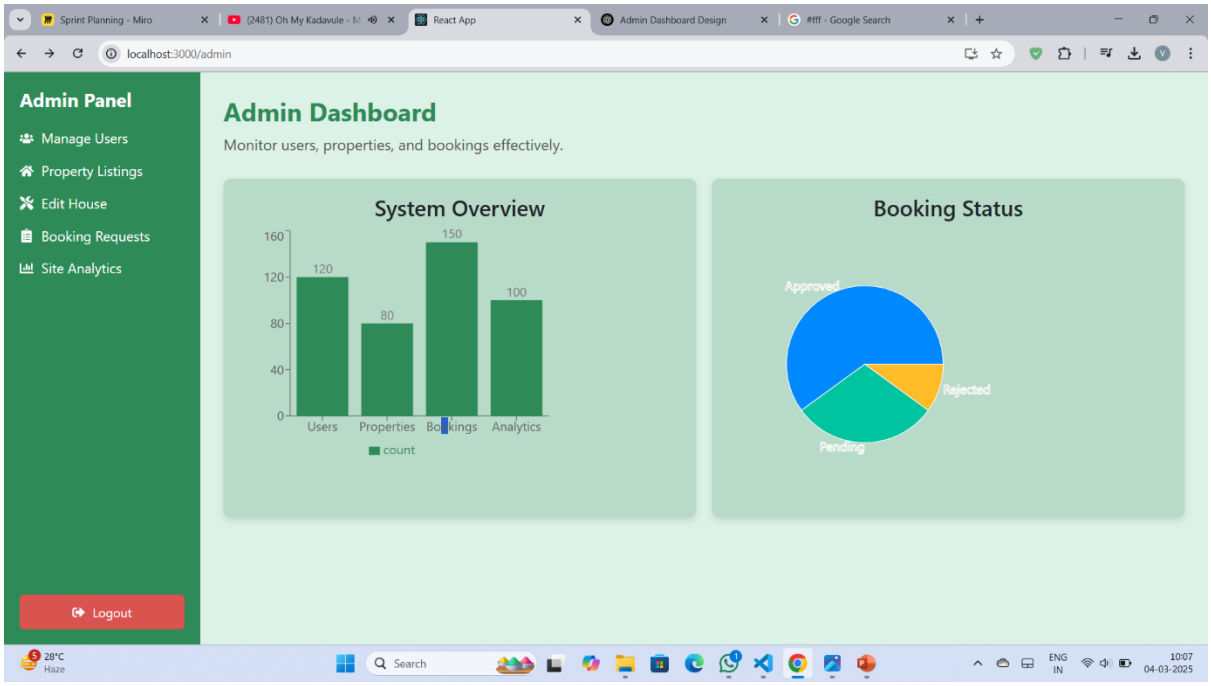
House Owner About Screen



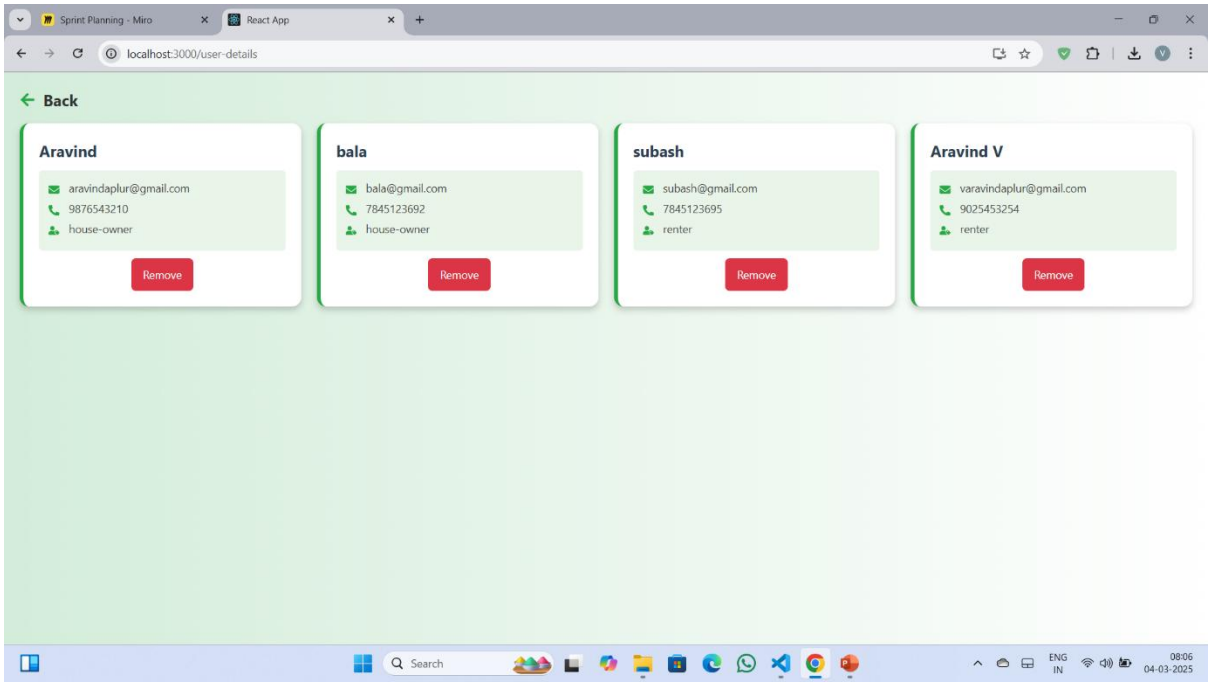
House Owner Contact Screen



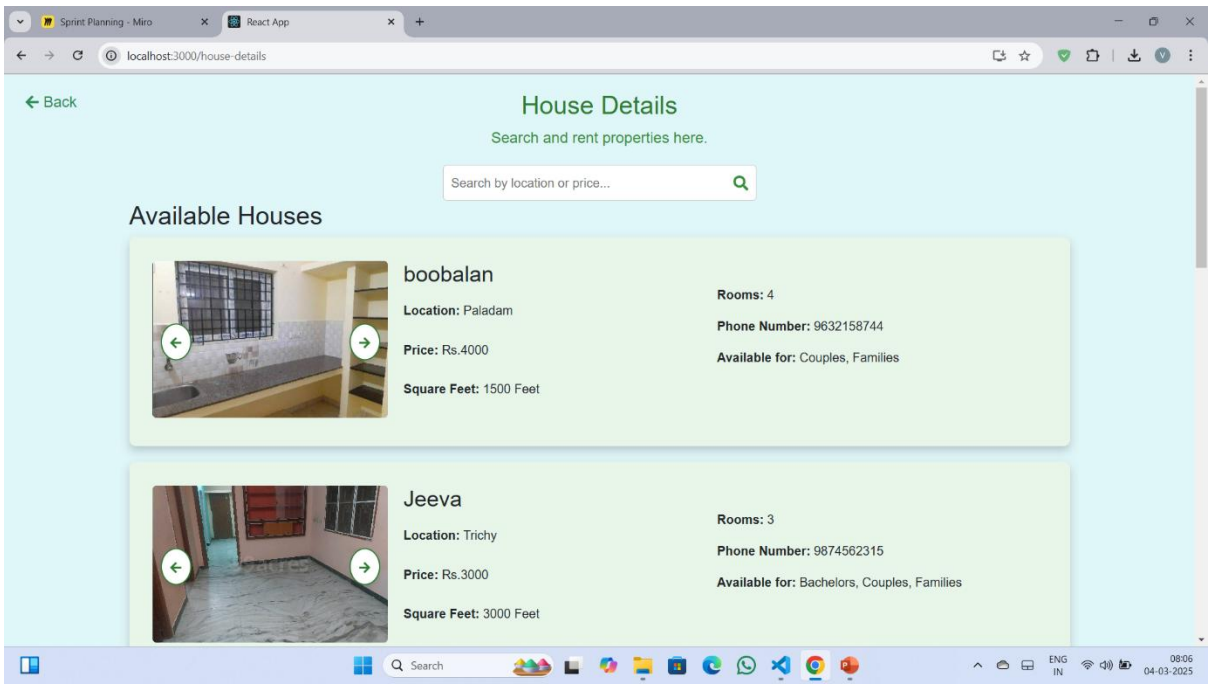
Admin Dashboard Screen



Admin User Details Screen



Admin House List Screen



Annexure-C – Source Code

FRONTEND:

App.jsx

```
import { useState, useEffect } from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { useSelector } from 'react-redux';
import { Toaster } from 'react-hot-toast';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';
import Departments from './pages/Departments';
import Subjects from './pages/Subjects';
import Teachers from './pages/Teachers';
import Students from './pages/Students';
import Appointments from './pages/Appointments';
import Chat from './pages/Chat';
import Videos from './pages/Videos';
import Profile from './pages/Profile';
import Attendance from './pages/Attendance';
import ProtectedRoute from './components/auth/ProtectedRoute';
import Layout from './components/layout/Layout';
import SocketListener from './components/common/SocketListener';

function App() {
  const { user } = useSelector((state) => state.auth);

  return (
    <Router>
      <div className="min-h-screen bg-secondary-50">
        <SocketListener />
        <Routes>
          <Route path="/login" element={!user ? <Login /> : <Navigate to="/" />} />
          <Route path="/register" element={!user ? <Register /> : <Navigate to="/" />} />
```

```
          <Route path="/" element={
            <ProtectedRoute>
              <Layout />
            </ProtectedRoute>
          }>
            <Route index element={<Dashboard />} />
            <Route path="departments" element={<Departments />} />
            <Route path="subjects" element={<Subjects />} />
            <Route path="teachers" element={<Teachers />} />
            <Route path="students" element={<Students />} />
            <Route path="appointments" element={<Appointments />} />
            <Route path="chat" element={<Chat />} />
            <Route path="chat/:userId" element={<Chat />} />
            <Route path="videos" element={<Videos />} />
            <Route path="profile" element={<Profile />} />
            <Route path="attendance" element={<Attendance />} />
          </Route>
        </Routes>
        <Toaster position="top-right" />
      </div>
    </Router>
  );
}

export default App;
```

BACKEND :

index.js :

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const cors = require('cors');
const helmet = require('helmet');
const path = require('path');
const morgan = require('morgan');
const socketio = require('socket.io');
const connectDB = require('./config/db');

const app = express();
const server = http.createServer(app);
const io = socketio(server, {
    cors: {
        origin: '*',
        methods: ['GET', 'POST']
    }
});

const authRoutes = require('./routes/authRoutes');
const departmentRoutes = require('./routes/departmentRoutes');
const subjectRoutes = require('./routes/subjectRoutes');
const teacherRoutes = require('./routes/teacherRoutes');
const appointmentRoutes = require('./routes/appointmentRoutes');
const messageRoutes = require('./routes/messageRoutes');
const notificationRoutes = require('./routes/notificationRoutes');
const aiRoutes = require('./routes/aiRoutes');
const videoRoutes = require('./routes/videoRoutes');
const studentRoutes = require('./routes/studentRoutes');
const registryRoutes = require('./routes/registryRoutes');
const attendanceRoutes = require('./routes/attendanceRoutes');
```

```
const uploadRoutes = require('./routes/uploadRoutes');

// Middleware
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ limit: '50mb', extended: true }));
const allowedOrigins = [
    'http://localhost:5173',
    'http://127.0.0.1:5173',
    'https://rvscas-portal.onrender.com'
];

app.use(cors({
    origin: function (origin, callback) {
        if (!origin) return callback(null, true);
        if (allowedOrigins.indexOf(origin) === -1) {
            return callback(null, true); //        }
        return callback(null, true);
    },
    credentials: true
}));
app.use(helmet({
    contentSecurityPolicy: false,
    crossOriginResourcePolicy: false,
}));
app.use(morgan('dev'));

const fs = require('fs');
const uploadsDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadsDir)) {
    fs.mkdirSync(uploadsDir);
}
app.use('/uploads', express.static(uploadsDir));
```

```
// Routes
app.use('/api/auth', authRoutes);
app.use('/api/departments', departmentRoutes);
app.use('/api/subjects', subjectRoutes);
app.use('/api/teachers', teacherRoutes);
app.use('/api/students', studentRoutes);
app.use('/api/appointments', appointmentRoutes);
app.use('/api/messages', messageRoutes);
app.use('/api/notifications', notificationRoutes);
app.use('/api/ai', aiRoutes);
app.use('/api/videos', videoRoutes);
app.use('/api/upload', uploadRoutes);
app.use('/api/registry', registryRoutes);
app.use('/api/attendance', attendanceRoutes);

app.get('/', (req, res) => {
    res.send('API is running...');
});

// Socket.io Connection
io.on('connection', (socket) => {
    console.log('New client connected:', socket.id);

    socket.on('join', (userId) => {
        socket.join(userId);
        socket.userId = userId;
        console.log(`User ${userId} joined their private room`);
    });

    socket.on('sendMessage', (data) => {
        const { receiverId, message } = data;
        io.to(receiverId).emit('newMessage', message);
    });
```

```
    // WebRTC Signaling Events
    socket.on('rtc-offer', (data) => {
        const { to, offer, senderName, callType } = data;
        const from = data.from || socket.userId;
        console.log(`[WebRTC] OFFER from ${from} to ${to} (Sender: ${senderName},
Type: ${callType || 'voice'})`);
        io.to(to).emit('rtc-offer', { from, offer, senderName, callType });
    });

    socket.on('rtc-answer', (data) => {
        const { to, answer } = data;
        const from = data.from || socket.userId;
        console.log(`[WebRTC] ANSWER from ${from} to ${to}`);
        io.to(to).emit('rtc-answer', { answer });
    });

    socket.on('rtc-candidate', (data) => {
        const { to, candidate } = data;
        const from = data.from || socket.userId;
        console.log(`[WebRTC] ICE CANDIDATE from ${from} to ${to}`);
        io.to(to).emit('rtc-candidate', { candidate });
    });

    socket.on('rtc-end', (data) => {
        const { to } = data;
        console.log(`[WebRTC] END CALL to ${to}`);
        io.to(to).emit('rtc-end');
    });

    socket.on('disconnect', () => {
        console.log('Client disconnected');
    });
});
```

```
// Error Handling Middleware
app.use((err, req, res, next) => {
    console.error('--- GLOBAL ERROR ---');
    console.error('Message:', err.message);
    if (err.code) console.error('Code:', err.code);
    console.error('Stack:', err.stack);

    const statusCode = res.statusCode === 200 ? 500 : res.statusCode;
    res.status(statusCode).json({
        message: err.message,
        stack: process.env.NODE_ENV === 'production' ? null : err.stack,
    });
});

const startServer = async () => {
    try {
        await connectDB();
        const PORT = process.env.PORT || 5006;
        const serverInstance = server.listen(PORT, '0.0.0.0', () => {
            console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on
port ${PORT}`);
        });
        serverInstance.timeout = 600000; // 10 minutes timeout for large uploads
    } catch (error) {
        console.error('Failed to start server:', error);
        process.exit(1);
    }
};

startServer();

// Make io accessible to routes
app.set('socketio', io);
```

db.js :

```
const mongoose = require('mongoose');
let gfs;

const connectDB = async () => {
    try {
        const conn = await mongoose.connect(process.env.MONGO_URI);
        console.log(`MongoDB Connected: ${conn.connection.host}`);
        return conn;
    } catch (error) {
        console.error(`Error: ${error.message}`);
        process.exit(1);
    }
};

module.exports = connectDB;
```